16. Code Review'de Senior'ları Etkilemek: Kendi Kodunun Şeytanının Avukatı Ol

PR'ı Ekibe Göndermeden Önce AI ile Sanal Code Review Simülasyonu
Bölüm 3 • Zorluk Derecesi: Orta Seviye • Tahmini Okuma Süresi: 12 dk

---

> 🎙️ **NotebookLM & Audio Overview Podcast Notları**
>
> **Podcast Tartışma Sorusu:**
> *"Kendi kodunuzu PR olarak açmadan önce yapay zekaya 'Beni acımasızca eleştir' dediğinizde yakaladığınız hatalar, kariyerinizi nasıl hızlandırır?"*
>
> **Bölümden Çıkarılacak 3 Altın Cümle:**
> - 📌 En iyi mühendisler kodlarını başkası incelemeden önce kendi kendilerine en sert eleştiriyi yapanlardır. - 📌 PR açmadan önce AI'a 'Kıdemli Güvenlik Mimarı' rolü vererek diff'i inceletmek basit hataları eler. - 📌 Code Review yorumlarına kişisel bir saldırı gibi değil; ücretsiz bir mentorluk fırsatı gibi yaklaşın.

📖 1. Giriş Hikayesi & Günlük Hayat Analojisi

Uzaya roket fırlatan bir NASA ekibini düşünün. Mühendisler roketi fırlatma rampasına koyup *"Hadi fırlatalım, bakalım havada ne olacak"* der mi?

Asla! Fırlatmadan önce 'Kırmızı Takım' (Red Team) adı verilen bir grup uzman gelir. Görevleri roketi kırmak, patlatmak ve en küçük zayıflığı bulmaktır. Bu testleri geçen roket güvenle uçar.

Yazılımda da Pull Request açmadan önce kendi 'Kırmızı Takım'ınızı kurabilirsiniz. Yapay zekaya: *"Sen şirketin en huysuz, en titiz kıdemli mimarısın. Bu koddaki tüm açıkları bul"* dediğinizde, gerçek code review'e çıktığınızda senior'lar kodunuzun mükemmelliğine hayran kalır.

🧠 2. AI Çağında Yeni Zihniyet Modeli

Pre-Flight (Uçuş Öncesi) Code Review Döngüsü:

```
[Yerel Kod Değişiklikleri]
           |
           ▼
[Adım 1: git diff Çıktısını Al]
           |
           ▼
[Adım 2: AI'a "Kıdemli Mimar Gözüyle İncele" Promptu Gönder]
           |
           ├── "Burada bellek sızıntısı var mı?"
           ├── "SQL Injection veya tip kaçağı var mı?"
           └── "İsimlendirmeler temiz kod standartlarına uyuyor mu?"
           |
           ▼
[Adım 3: Düzeltmeleri Yap & Testleri Çalıştır]
           |
           ▼
[Adım 4: Kusursuz PR'ı Takıma Sun]
```

🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt)

Aşağıdaki prompt şablonunu Cursor, Claude Code, ChatGPT veya Copilot ortamınızda doğrudan kullanabilirsiniz:

````
# ROL: Titiz ve Huysuz Kıdemli Yazılım Mimarı
Ben ekibinde çalışan bir junior mühendisim. Aşağıda hazırladığım Pull Request diff'i
yer alıyor.

## GÖREV:
Bana kibar davranma; koddaki teknik eksiklikleri acımasızca ortaya çıkar:
1. Hangi sınır durum (edge case) gözden kaçmış?
2. Performans veya bellek tüketimi açısından nerede patlayabilir?
3. Kod okunabilirliği ve SOLID prensipleri açısından hangi 2 kritik eleştirin var?

## DIFF İÇERİĞİ:
```typescript
export async function updateUserEmail(userId: string, newEmail: string) {
  const user = await db.user.findUnique({ where: { id: userId } });
  user.email = newEmail;
  await db.user.update({ where: { id: userId }, data: { email: newEmail } });
  await sendVerificationEmail(newEmail);
  return { success: true };
}
````
````

```
---

## 💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi

#### ❌ Acemi / Kontrolsüz AI Çıktısı (Riskli):
```typescript
// Pre-Flight Öncesi: E-posta formatı kontrol edilmemiş, mükerrer kontrolü yok,
transaction yok!
export async function updateUserEmailNaive(userId: string, newEmail: string) {
  // Eğer newEmail başka bir kullanıcıda zaten varsa veritabanı çöker!
  // Eğer e-posta servisi hata verirse DB güncellenmiş kalır (inconsistent state)!
  return await db.user.update({ where: { id: userId }, data: { email: newEmail } });
}
```

✅ Kıdemli Mühendis Tarafından İyileştirilmiş Sürüm (Güvenli & Temiz):

```typescript
import { z } from "zod";

const EmailSchema = z.string().email("Geçersiz e-posta formatı").toLowerCase();

export async function updateUserEmailRobust(userId: string, rawEmail: string) {
  // 1. Giriş Doğrulaması
  const validatedEmail = EmailSchema.parse(rawEmail);

  // 2. İşlem bütünlüğü (Database Transaction)
  return await db.$transaction(async (tx) => {
    // Çakışma kontrolü
    const existing = await tx.user.findUnique({ where: { email: validatedEmail } });
    if (existing && existing.id !== userId) {
      throw new ConflictError("Bu e-posta adresi başka bir hesap tarafından
kullanılıyor.");
    }

    const updatedUser = await tx.user.update({
      where: { id: userId },
      data: { email: validatedEmail, emailVerified: false }
    });

    // 3. Yan etki (Outbox deseni veya güvenli e-posta kuyruğu)
    await emailQueue.add("send-verification", { userId, email: validatedEmail });

    return { success: true, user: updatedUser };
  });
}
```

💡 **Kod Analizi:** Sanal review sonrasında: E-posta normalizasyonu (lowercase), mükerrer kayıt kontrolü, `db.$transaction` ile veri tutarlılığı ve doğrudan API çağırmak yerine kuyruk (queue) kullanımı eklenmiştir.
```

⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı

- **Tuzak:** Code review'de gelen eleştirilere savunmaya geçip kırılmak.
  - **Çözüm:** Eleştirilen siz değilsiniz, koddur. Her yorumu öğrenme fırsatı olarak görün.
- **Tuzak:** AI pre-flight review yapmadan doğrudan PR açıp basit hatalarla senior'ın vaktini almak.
  - **Çözüm:** Her PR öncesi en az 1 kez AI'a diff kontrolü yaptırın.
- **Tuzak:** Review yorumunu uygulayıp testleri tekrar çalıştırmadan push etmek.
  - **Çözüm:** Her review revizyonundan sonra tüm test paketini yeniden çalıştırın.

🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri

📽️ Ders Başlığı: *Ders 16: Code Review Başarısı - Kendi Kodunun Şeytanının Avukatı Olmak*

- 🖥️ **Slayt 1:** Neden en iyi mühendisler kendi kodlarını acımasızca eleştirir?
- 🖥️ **Slayt 2:** Pre-Flight Check: AI ile Sanal Kıdemli Mimar Simülasyonu.
- 🖥️ **Slayt 3:** Review yorumlarına profesyonel yanıt verme sanatı.
- 🖥️ **Slayt 4:** Veri tutarlılığı: Database Transaction ve Hata Sınırları.

Canlı Kodlama / Ekran Kaydı Senaryosu:
Basit bir veritabanı güncelleme fonksiyonunu AI'a verin. 'Bu koddaki race condition ve transaction açıklarını bul' deyin. AI'ın sunduğu eleştirileri inceleyip kodu refactor edin.

🎯 7. Günün Uygulamalı Görevi & Mini Sınav

🛠️ Günün Görevi:

Bugün yazacağınız bir kodu GitHub'a push etmeden önce AI'a 'Sen bir güvenlik müfettişisin, bu diff'i eleştir' diyerek taratın ve bulduğu en az 1 noktayı düzeltin.

🧠 Bölüm Sonu Kontrol Testi:

Soru 1: Bir veritabanı işleminde birden fazla ilişkili adımın ya hep ya hiç (atomik) olarak çalışmasını garanti eden mekanizma nedir? - A) CSS Flexbox - B) Database Transaction ($transaction) - C) setTimeout fonksiyonu - D) console.warn > **Doğru Yanıt:** B > **Açıklama:** Veritabanı işlemleri (Transaction), içindeki adımlardan biri başarısız olduğunda tüm değişiklikleri geri alarak (rollback) veri tutarsızlığını önler.

Soru 2: Code Review sürecinde gelen teknik bir eleştiriye profesyonel bir junior mühendis nasıl yaklaşmalıdır? - A) Eleştiriyi kişisel bir hakaret sayıp PR'ı silmelidir - B) Eleştirinin arkasındaki mimari nedeni anlamaya çalışıp yapıcı şekilde tartışmalı veya kodu iyileştirmelidir - C) Yorum yapan senior'ı

şikayet etmelidir - D) Yorumları görmezden gelip PR'ı doğrudan merge etmelidir > **Doğru Yanıt:** B > **Açıklama:** Code Review, yazılım mühendisliğinde en hızlı öğrenme ve mentorluk kanalıdır; yapıcı yaklaşım profesyonelliğin temelidir.